Decoupled consensus

1 Model

Validators. A fixed set V of validators is given, and each validator i has a fixed positive integer weight $w(i)$. For $S \subseteq V$ write $w(S) = \sum_{i \in S} w(i)$, and let $W = w(V)$. Two thresholds are used throughout:

$$q = \left\lceil \frac{2W}{3} \right\rceil, \qquad m = \left\lfloor \frac{W}{2} \right\rfloor + 1.$$

A *quorum* is a set of validators of weight at least q. A *majority* is a set M of weight at least m.

Slots and rounds. Time is divided into consecutive *slots*, numbered from 0. A *round* is a group of R consecutive slots, for a fixed protocol constant $R \geq 1$: round r consists of slots $rR, \ldots, rR + R - 1$, and its first slot is the round's *opening slot*. Write $\mathsf{round}(s) = \lfloor s/R \rfloor$ for the round of slot s.

Definition 1 (Blocks and chains)**.** The *genesis block* B_{gen} has no parent and slot 0. Every other block has a *parent* block, a *slot* greater than its parent's slot, a set of attestations (Definition 3), and a *proposal root* $\mathsf{proposal\_root}$, a block or $\perp$. A block whose slot is a round's opening slot is an *opening block*; the proposal root is read only in opening blocks (Section 4). Each slot has one assigned proposer, and a block is signed by its slot's proposer; how proposers are assigned is outside this document's scope.

A *chain* is the path from B_{gen} to a block B through parent links; chains are in bijection with their tips, and we identify the two. Blocks form a tree rooted at B_{gen}. Write $B \preceq C$ when $B = C$ or B is an ancestor of C, and $B \prec C$ for strict ancestry; C is a *descendant* of B when $B \preceq C$, so descendant is reflexive. Blocks B and C are *compatible* when $B \preceq C$ or $C \preceq B$, and *conflict* otherwise.

Definition 2 (Height)**.** Height is the finality counter, separate from slot. Genesis counts as justified and finalized at height 0, and every chain begins at height 1. Height advances by exactly one, at a justification or a progress event (Section 2).

Definition 3 (Attestation)**.** An *attestation* is a tuple

$$(\mathsf{validator}, \mathsf{round}, \mathsf{head}, \mathsf{height}, \mathsf{target}, \mathsf{finalize\_height}, \mathsf{finalize\_target})$$

signed by $\mathsf{validator} \in V$, where round names the round the attestation belongs to. The *head* is a block or $\perp$ (Section 4). The *height pair* $(\mathsf{height}, \mathsf{target})$ is a *target vote* (h, T) with $T \neq \perp$, an *empty-target vote* $(h, \perp)$, or the empty pair $(\perp, \perp)$. The *finality pair* is a pair (h_f, T_f) with $T_f \neq \perp$, or the empty pair.

Definition 4 (Goldfish vote)**.** Each slot has a *committee*, a known subset of V; how committees are drawn is outside this document's scope. A *Goldfish vote* is a tuple $(\mathsf{index}, \mathsf{slot}, \mathsf{block})$, signed by the validator at index index in that slot's committee. Goldfish votes are counted by committee index, one unit each, independently of weight.

Definition 5 (Slashing conditions)**.** Two signed attestations by one validator are slashable when either condition holds:

E1: one attestation's finality pair is (h, T) with $T \neq \bot$, and the other's height pair is (h, T') with $T' \neq T$, including $T' = \bot$;

E2: the two height pairs are (h, T) and (h, T') with $T, T' \neq \bot$ and $T \neq T'$.

The two attestations need not be distinct: the conflicting pairs may occur in one attestation. An empty height pair has height $\bot$ and triggers neither condition.

2 State transition

This section defines the chain state and its per-block transition. A block is evaluated from its parent's post-state: the post-state of block B is $\sigma[B] = \textsc{state\_transition}(\sigma[B.\mathsf{parent}], B)$ (Figure 1), a deterministic function of B's chain, so every node computes the same value. The *state-height* of B is $\sigma[B].h$.

Definition 6 (Chain state)**.** A chain state is

$$\sigma = (L,\ s,\ h,\ T_h,\ \mathsf{target\_participation},\ \mathsf{progress},\ \mathsf{finalize},\ J,\ h_j,\ F,\ h_F),$$

where L is the latest block, s is the current slot, h is the current height, and T_h is the current height's *target*: the block that carried the transition into h (genesis for height 1). The arrays target_participation, progress, and finalize hold one bit per validator, and the quorum sets

$$Q_{\text{target}}(\sigma) = \{i : \sigma.\mathsf{target\_participation}[i]\},\ Q_{\text{prog}}(\sigma) = \{i : \sigma.\mathsf{progress}[i]\},\ Q_{\text{finality}}(\sigma) = \{i : \sigma.\mathsf{finalize}[i]\}$$

are derived from them when used. The pair (J, h_j) is the latest justified block and its height, and (F, h_F) the latest finalized block and its height. The genesis state has $L = T_h = B_{\text{gen}}$, $s = 0$, $h = 1$, $(J, h_j) = (F, h_F) = (B_{\text{gen}}, 0)$, and every bit false.

Attestation processing. Each attestation a in a block is classified against the block's prestate σ (Figure 1):

- a's height pair is the exact pair $(\sigma.h, \sigma.T_h)$: set $\sigma.\mathsf{target\_participation}[a.\mathsf{validator}]$;
- a's height pair is $(\sigma.h, \bot)$, or is $(\sigma.h, T)$ with $T \preceq \sigma.L$ — a current-height vote already on the chain: set $\sigma.\mathsf{progress}[a.\mathsf{validator}]$;
- a's finality pair is $(\sigma.h_j, \sigma.J)$ — a commitment to the latest justification: set $\sigma.\mathsf{finalize}[a.\mathsf{validator}]$.

Note that an exact target vote sets both the target_participation and progress bits, contributing both to a justification and to height progress together with any empty-target votes.

Height events. After a block's attestations are folded in, the height events are checked once, in order ($K \geq 2$ and $D \geq 1$ are protocol constants):

1. *finalize*: if $\sigma.h_j > \sigma.h_F$ and $w(Q_{\text{finality}}(\sigma)) \geq q$, set $(\sigma.F, \sigma.h_F) \leftarrow (\sigma.J, \sigma.h_j)$;
2. *justify*: if $\neg\big((K \mid \sigma.h) \wedge (\sigma.h - \sigma.h_F > D)\big)$ and $w(Q_{\text{target}}(\sigma)) \geq q$, set $(\sigma.J, \sigma.h_j) \leftarrow (\sigma.T_h, \sigma.h)$, clear $\sigma.\mathsf{finalize}$, and advance to new height;
3. *progress*: otherwise, if $w(Q_{\text{prog}}(\sigma)) \geq q$, advance to new height without a new justification.

An advance increments $\sigma.h$, records the advancing block as the new height's target, and clears both height-participation arrays. A new justification restarts the finality tally. Height events are checked only at blocks; a slot without a block changes no state.

Figure 1: State transition

```
1: function STATE_TRANSITION(σ, B)                                   ▷ σ = σ[B.parent]
2:     σ.s ← B.slot
3:     for all a ∈ B.attestations do
4:         σ ← PROCESS_ATTESTATION(σ, a)
5:     σ.L ← B
6:     return PROCESS_HEIGHT_EVENTS(σ)
7: function PROCESS_ATTESTATION(σ, a)
8:     i ← a.validator
9:     if a's finality pair = (σ.h_j, σ.J) then
10:        σ.finalize[i] ← true
11:    if a's height pair = (σ.h, σ.T_h) then
12:        σ.target_participation[i] ← true
13:    if a's height pair = (σ.h, ⊥), or = (σ.h, T) with T ⪯ σ.L then
14:        σ.progress[i] ← true
15:    return σ
16: function PROCESS_HEIGHT_EVENTS(σ)
17:    if σ.h_j > σ.h_F and w(Q_finality(σ)) ≥ q then
18:        (σ.F, σ.h_F) ← (σ.J, σ.h_j)
19:    if ¬((K | σ.h) ∧ (σ.h − σ.h_F > D)) and w(Q_target(σ)) ≥ q then
20:        (σ.J, σ.h_j) ← (σ.T_h, σ.h);   σ.finalize ← false^V
21:        return ADVANCE_HEIGHT(σ)
22:    if w(Q_prog(σ)) ≥ q then
23:        return ADVANCE_HEIGHT(σ)
24:    return σ
25: function ADVANCE_HEIGHT(σ)
26:    σ.h ← σ.h + 1;   σ.T_h ← σ.L
27:    σ.target_participation, σ.progress ← false^V
28:    return σ
```

Nonjustifiable heights. During prolonged nonfinality, in particular while finality lags more than D heights behind, every K-th height becomes "nonjustifiable": the justify event requires $(K \mid \sigma.h) \wedge (\sigma.h - \sigma.h_F > D)$ to be false. As we see later, this is to enable recovery after periods of asynchrony. We can still move past such heights through the progress event.

3 Finality store

A node tracks the blocks it has accepted in a local *store*, which also maintains the finality values that root its fork choice.

Definition 7 (Store)**.** A store is

$$\Sigma = (s,\ \mathcal{T},\ \sigma[\cdot],\ F,\ J,\ h_j,\ h_{\max}),$$

where $\Sigma.s$ is the current slot, $\Sigma.\mathcal{T}$ is the tree of accepted blocks, and $\Sigma.\sigma[B]$ is the post-state of each $B \in \Sigma.\mathcal{T}$. $\Sigma.F$ is the finalized block the store has adopted, and $(\Sigma.J, \Sigma.h_j)$ the justified pair rooting its fork choice; both blocks are always accepted. $\Sigma.h_{\max}$ is the greatest state-height the store has ever

accepted. The genesis store has $s = 0$, $\mathcal{T} = \{B_{\text{gen}}\}$, $\sigma[B_{\text{gen}}]$ the genesis state (Definition 6), $F = B_{\text{gen}}$, $(J, h_j) = (B_{\text{gen}}, 0)$, and $h_{\max} = 1$.

Definition 8 (Viable subtree, root, and head)**.** A *leaf* of $\Sigma.\mathcal{T}$ is a block without accepted children. A block is *viable* when some leaf L descending from it satisfies $\Sigma.\sigma[L].h \geq \Sigma.h_{\max} - 1$: its branch reaches within one height of the store's frontier. Viability is inherited by ancestors, so the set $\mathcal{V}(\Sigma)$ of viable blocks is prefix-closed. The *fork-choice root* is $\Sigma.J$ when $\Sigma.h_{\max} = \Sigma.h_j + 1$, and $\Sigma.F$ otherwise; the *candidate tree* $\mathcal{C}(\Sigma)$ is the tree of viable blocks rooted at the fork-choice root. Fork choice selects a head within the candidate tree,

$$\text{GET\_HEAD}(\Sigma,\ \Omega) \in \mathcal{C}(\Sigma),$$

where Ω is the available-chain data that selects among the candidates; Sections 4 and 5 define it, and record one exception for a walk root fixed earlier in a round.

Store maintenance. The store changes through two handlers (Figure 2). $\text{ON\_SLOT}(\Sigma, s)$ advances the store's slot at the start of each slot, before any block of that slot is processed. $\text{ON\_BLOCK}(\Sigma, B)$ accepts a block only if its slot has started, its parent is accepted, and B descends from $\Sigma.F$. For an accepted block, the post-state $\Sigma.\sigma[B]$ is computed, stored, and offered to the update rules. A block that fails admission is not added to $\Sigma.\mathcal{T}$, but it and every other received signed object remain available as evidence. $\text{PROCESS\_UPDATES}(\Sigma, \sigma)$ folds an offered post-state into the store: it raises $\Sigma.h_{\max}$, replaces the justified pair when the offer dominates in (h, hash) order and descends from $\Sigma.F$, and advances the finalized block only to a viable proper descendant of $\Sigma.F$ below $\Sigma.J$. When finality advances, the store prunes from $\Sigma.\mathcal{T}$ every block conflicting with the new $\Sigma.F$; the pruned blocks remain available as signed evidence.

By construction, $\Sigma.F$ only ever moves to a proper descendant of itself — the store never reverts a finalization — $\Sigma.F \preceq \Sigma.J$ holds at all times, every block in $\Sigma.\mathcal{T}$ is compatible with $\Sigma.F$, $\Sigma.h_{\max}$ never decreases, and every candidate descends from $\Sigma.F$.

4 Rounds, grades, and round roots

Taking the start of slot 0 as time 0, slot s starts at time $4\Delta s$, where Δ strictly bounds delivery between honest validators: an object sent at time t reaches every honest validator before $t + \Delta$. All scheduled times are multiples of Δ, and every validator executes every phase once, at its public time. Messages are processed on receipt; a message received exactly at a scheduled instant is processed after that instant's tick.

Definition 9 (Schedule)**.** A slot starting at time t runs four phases:

$$t \text{ proposal}, \quad t + \Delta \text{ vote}, \quad t + 2\Delta \text{ support freeze}, \quad t + 3\Delta \text{ slot-view freeze},$$

and the slot's available confirmation is evaluated 2Δ into the following slot (Section 5). For round r, write t_r for the proposal time of its opening slot. The round's *action time* is

$$a_r = t_r + 6\Delta,$$

the evaluation time of its opening slot; there the validator performs the round's SG and FG action (Section 6). Round r also has four *grade instants*, at which it grades the received round-$(r-1)$ attestations:

$$\Gamma^{-1} : t_r - \Delta, \qquad \Gamma^0 : t_r, \qquad \Gamma^1 : t_r + \Delta, \qquad \Gamma^2 : t_r + 2\Delta$$

(Section 4). The action lies in the round's second slot, and the heads signed at a_r must arrive before round $r+1$'s first grade instant — $a_r + \Delta \leq t_{r+1} - \Delta$ — so this schedule requires $R \geq 2$.

Figure 2: Finality store

```
1: function ON_SLOT(Σ, s)                                  ▷ At the start of slot s, before its blocks
2:     Σ.s ← s
3:     return Σ
4: function ON_BLOCK(Σ, B)
5:     if B is an opening block and Σ.root_proposal[round(B.slot)] is unset then
6:         Σ.root_proposal[round(B.slot)] ← B.proposal_root                          ▷ Section 4
7:     if B.slot > Σ.s or B.parent ∉ Σ.T or Σ.F ⋠ B then
8:         return Σ
9:     Σ.σ[B] ← STATE_TRANSITION(Σ.σ[B.parent], B)
10:    Σ.T ← Σ.T ∪ {B}
11:    return PROCESS_UPDATES(Σ, Σ.σ[B])
12: function PROCESS_UPDATES(Σ, σ)
13:    Σ.h_max ← max(Σ.h_max, σ.h)
14:    if Σ.F ⪯ σ.J and (σ.h_j, hash(σ.J)) > (Σ.h_j, hash(Σ.J)) then
15:        (Σ.J, Σ.h_j) ← (σ.J, σ.h_j)
16:    if Σ.F ≺ σ.F and σ.F ⪯ Σ.J and σ.F ∈ V(Σ) then
17:        Σ.F ← σ.F
18:        Σ.T ← {B ∈ Σ.T : B is compatible with Σ.F}
19:    return Σ
20: function FORK_CHOICE_ROOT(Σ)
21:    if Σ.h_max = Σ.h_j + 1 then
22:        return Σ.J
23:    return Σ.F
24: function GET_HEAD(Σ, Ω)
25:    return a block in C(Σ), selected by Ω
```

Definition 10 (Timed store)**.** A validator's timed store contains the finality-store fields of Section 3, a clock, per-round head bookkeeping, and per-round grade and root bookkeeping:

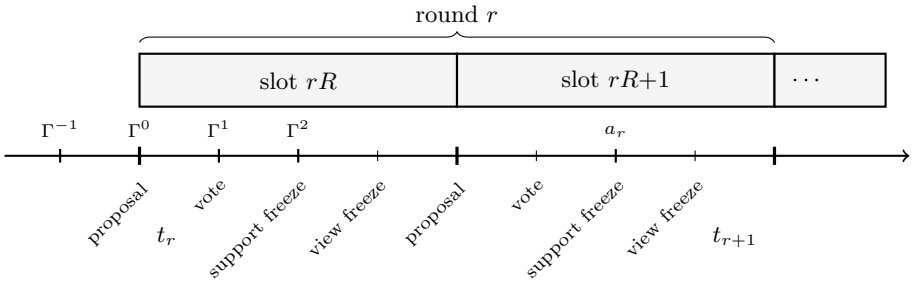
$$\Sigma \mathrel{+}= (t,\ \mathsf{head}[\cdot],\ \mathsf{equiv}[\cdot],\ \mathsf{root\_proposal}[\cdot],\ \mathsf{sg\_root}[\cdot],\ \mathsf{action\_root}[\cdot]).$$

$\Sigma.t$ is the current time: ON_TICK is invoked as time passes, including once at every scheduled instant (Figure 6). It sets $\Sigma.t$ to the clock and runs the phase scheduled at that instant. An object's handler runs only after its dependencies, including any block ancestry, have been received; until then the object waits.

At Γ^0, the opening proposer proposes its block (Section 6). At Γ^1, each validator stores the SG root in $\Sigma.\mathsf{sg\_root}[r]$; $\Sigma.\mathsf{root\_proposal}[r]$ is registered earlier, by ON_BLOCK, when the round's first opening block arrives (Definition 13). At a_r, it stores the action root in $\Sigma.\mathsf{action\_root}[r]$. These entries are fixed after their scheduled writes.

ON_ATTESTATION ignores an empty head; it keeps, per round and validator, the first processed nonempty head with its processing time, and the time at which a different head from the same validator was first processed ($\Sigma.\mathsf{equiv}$). The support scores of Definition 11 read these entries against the grade instants. Initially the per-round maps and sets are empty; the scheduled hooks fill each round's entries.

Figure 3: One round of the schedule, drawn for $R = 2$. Above the axis: the round's distinguished times — the grade instants $\Gamma^{-1}, \Gamma^0, \Gamma^1, \Gamma^2$ grade the round-$(r-1)$ attestations, and the SG/FG action at a_r follows the opening slot's confirmation evaluation. Below: the phases of each slot.



Round r grades the heads of the round-$(r-1)$ attestations, through the head bookkeeping of Definition 10, and uses the grades to fix one SG root per round: the proposer offers a root in its opening block, each receiver checks it, and the accepted or fallback root anchors the round's Goldfish votes (Section 5) and its SG and FG outputs (Section 6).

Definition 11 (Support)**.** Fix round r, block B, and $j \in \{-1, 0, 1, 2\}$. Read the round-$(r-1)$ entries of the timed store and define

$$\mathcal{H}_j(B) = \{i \in V : \Sigma.\mathsf{head}[r-1][i] = (H, t_H),\ t_H < \Gamma^j,\ B \preceq H\},$$

$$\mathcal{E}_j = \{i \in \Sigma.\mathsf{equiv}[r-1] : \Sigma.\mathsf{equiv}[r-1][i] < \Gamma^j\}.$$

Thus $\mathcal{H}_j(B)$ contains the validators whose stored head was processed before Γ^j and supports B, and $\mathcal{E}_j$ contains the validators whose stored equivocation time is before that instant. For round 0 the round-(-1) entries are empty, so every score is 0. The support scores are

$$S_j(B) = w(\mathcal{H}_j(B) \setminus \mathcal{E}_j), \qquad \overline{S}_j(B) = w(\mathcal{H}_j(B) \cup \mathcal{E}_j), \qquad S_{-1,1}(B) = w(\mathcal{H}_{-1}(B) \setminus \mathcal{E}_1).$$

Γ^{-1} supplies the head cutoff in $S_{-1,1}$; its equivocation cutoff is Γ^1, so Figure 6 writes grade 3 at Γ^1.

Definition 12 (Grades)**.** For every block B,

$$\mathsf{G3}(B) \Leftrightarrow S_{-1,1}(B) \geq m, \quad \mathsf{G2}(B) \Leftrightarrow S_0(B) \geq m, \quad \mathsf{G1}(B) \Leftrightarrow \overline{S}_1(B) \geq m, \quad \mathsf{G0}(B) \Leftrightarrow \overline{S}_2(B) \geq m.$$

A grade is computed when a rule tests it (Figure 4): the scores compare stored processing times against past instants, so a grade evaluated at any time after its instants gives one fixed answer. Support counts descendants, so every grade is inherited by ancestors; and two conflicting blocks cannot both hold direct support m, so the blocks with grade 2, and those with grade 3, lie on one chain. Grades are signed evidence. The rules below use them only with their stated candidate-tree and path conditions.

The score expressions in Figure 6 are evaluated from the current store by Figure 4 at that tick.

Figure 4: Support scores in round r

```
1: function SUPPORT_SCORES(Σ, r, j, B)
2:     H_j(B) ← {i ∈ V : Σ.head[r−1][i] = (H, t_H),  t_H < Γ^j,  B ⪯ H}
3:     E_j ← {i ∈ Σ.equiv[r−1] : Σ.equiv[r−1][i] < Γ^j}
4:     return (w(H_j(B) \ E_j),  w(H_j(B) ∪ E_j))                ▷ S_j(B), S̄_j(B)
5: function TWO_VIEW_SUPPORT(Σ, r, B)
6:     H_{−1}(B) ← {i ∈ V : Σ.head[r−1][i] = (H, t_H),  t_H < Γ^{−1},  B ⪯ H}
7:     E_1 ← {i ∈ Σ.equiv[r−1] : Σ.equiv[r−1][i] < Γ^1}
8:     return w(H_{−1}(B) \ E_1)                                  ▷ S_{−1,1}(B)
```

Proposal root. At t_r, the round's opening proposer builds its opening block (Section 6) and sets the block's proposal-root field to GET_PROPOSAL_ROOT(Σ, r) (Figure 5): the deepest block in $\mathcal{C}(\Sigma)$ with grade 2, or the current fork-choice root when no such block exists.

Definition 13 (Root proposal)**.** The round's *root proposal* $\Sigma.\mathsf{root\_proposal}[r]$ is the proposal root of the first round-r opening block that ON_BLOCK processes (Figure 2); later opening blocks of the round are ignored, as they would be at the gossip level. It is $\perp$ while no opening block has arrived, and only the opening slot's vote time reads it, so a later arrival has no effect.

Definition 14 (SG root)**.** At the opening slot's vote time, a validator derives its round's SG root. Let C be its fork-choice root and its *lower root* R_{low} the deepest block in $\mathcal{C}(\Sigma)$ with grade 3 strictly descending from C, or C itself when there is none. Let $R_{\text{prop}} = \Sigma.\mathsf{root\_proposal}[r]$, the round's root proposal ($\perp$ when none).

The validator accepts the proposed root exactly when

$$R_{\text{low}} \preceq R_{\text{prop}}, \qquad R_{\text{prop}} \in \mathcal{C}(\Sigma), \qquad \mathsf{G1}(R_{\text{prop}}).$$

On acceptance, the *SG root* R_{SG} is R_{prop}; otherwise it is R_{low}. ON_TICK stores R_{SG} in $\Sigma.\mathsf{sg\_root}[r]$ at this vote time; the lower root is not retained.

Each Goldfish fork choice starts from its current fork-choice root C. If $C \preceq R_{\text{SG}}$, it first moves to R_{SG}; otherwise this step does not occur. The opening proposer applies the same rule with its chosen proposal root in place of R_{SG} (Figure 5).

Definition 15 (Action root)**.** At a_r, the validator derives the root anchoring its SG and FG outputs, GET_ACTION_ROOT(Σ, r), and ON_TICK stores it in $\Sigma.\mathsf{action\_root}[r]$ (Figure 5). The walk root anchors the outputs when it is in $\mathcal{C}(\Sigma)$ and either equals the fork-choice root or has grade 1; otherwise the fork-choice root itself does — a validator's own selection needs no external backing, while a round root adopted from others must still be viable and majority-backed at signing time.

The functions of Figure 5 read the stored round entries and compute grades as needed.

Figure 5: Round-root functions called by ON_TICK

```
1: function GET_PROPOSAL_ROOT(Σ, r)                                   ▷ Proposer, at t_r
2:     G ← {B ∈ C(Σ) : G2(B)}
3:     if G ≠ ∅ then
4:         return the deepest block in G
5:     return FORK_CHOICE_ROOT(Σ)
6: function GET_LOWER_ROOT(Σ, r)
7:     G ← {B ∈ C(Σ) : G3(B), FORK_CHOICE_ROOT(Σ) ≺ B}
8:     if G ≠ ∅ then
9:         return the deepest block in G
10:    return FORK_CHOICE_ROOT(Σ)
11: function GET_SG_ROOT(Σ, r)
12:    R_low ← GET_LOWER_ROOT(Σ, r)
13:    if Σ.root_proposal[r] = ⊥ then
14:        return R_low
15:    R_prop ← Σ.root_proposal[r]
16:    if R_low ⪯ R_prop and R_prop ∈ C(Σ) and G1(R_prop) then
17:        return R_prop
18:    return R_low
19: function GET_WALK_ROOT(Σ, r)
20:    C ← FORK_CHOICE_ROOT(Σ)
21:    if C ⪯ Σ.sg_root[r] then
22:        return Σ.sg_root[r]
23:    return C
24: function GET_ACTION_ROOT(Σ, r)                                    ▷ At a_r
25:    C ← FORK_CHOICE_ROOT(Σ)
26:    R ← GET_WALK_ROOT(Σ, r)
27:    if R ∈ C(Σ) and (R = C or G1(R)) then
28:        return R
29:    return C
```

Figure 6: Timed store

```
1: function ON_TICK(Σ, t)   ▷ Called whenever time advances; scheduled instants are not skipped
2:     assert Σ.t < t;   Σ.t ← t
3:     if t = 4Δs for some slot s then
4:         Σ ← ON_SLOT(Σ, s)
5:     if t = t_r for some round r and this validator is the opening proposer then
6:         PROPOSE_BLOCK(Σ)                                                    ▷ Section 6
7:     if t = t_r + Δ for some round r then
8:         Σ.sg_root[r] ← GET_SG_ROOT(Σ, r)                                    ▷ Figure 5
9:     if t = a_r for some round r then
10:        Σ.action_root[r] ← GET_ACTION_ROOT(Σ, r)                            ▷ Figure 5
11:    return Σ
12: function ON_ATTESTATION(Σ, α)                          ▷ Received directly or in a block
13:    i ← α.validator;   r ← α.round
14:    if α.head = ⊥ then
15:        return Σ
16:    if i ∉ Σ.head[r] then
17:        Σ.head[r][i] ← (α.head, Σ.t)
18:    else if α.head ≠ Σ.head[r][i]'s head and i ∉ Σ.equiv[r] then
19:        Σ.equiv[r][i] ← Σ.t
20:    return Σ
```

5 Goldfish voting and confirmation

[To be drafted.]

6 Honest validator spec

[To be drafted.]